Random Forest Project

For this project I explored publicly available data from LendingClub.com. Lending Club connects people who need money (borrowers) with people who have money (investors). As an investor I would want to invest in people who showed a profile of having a high probability of paying me back. I created a model that helped predict this.

I used lending data from 2007-2010 to classify and to predict whether or not the borrower paid back their loan in full. Here are what the columns represent:

- credit.policy: 1 if the customer meets the credit underwriting criteria of LendingClub.com, and 0 otherwise.
- purpose: The purpose of the loan (takes values "credit_card", "debt_consolidation", "educational", "major_purchase", "small_business", and "all_other").
- int.rate: The interest rate of the loan, as a proportion (a rate of 11% would be stored as 0.11). Borrowers judged by LendingClub.com to be more risky are assigned higher interest rates.
- installment: The monthly installments owed by the borrower if the loan is funded.
- log.annual.inc: The natural log of the self-reported annual income of the borrower.
- dti: The debt-to-income ratio of the borrower (amount of debt divided by annual income).
- fico: The FICO credit score of the borrower.
- days.with.cr.line: The number of days the borrower has had a credit line.
- revol.bal: The borrower's revolving balance (amount unpaid at the end of the credit card billing cycle).
- revol.util: The borrower's revolving line utilization rate (the amount of the credit line used relative to total credit available).
- inq.last.6mths: The borrower's number of inquiries by creditors in the last 6 months.
- delinq.2yrs: The number of times the borrower had been 30+ days past due on a payment in the past 2 years.
- pub.rec: The borrower's number of derogatory public records (bankruptcy filings, tax liens, or judgments).

Import Libraries

Import the usual libraries for pandas and plotting. You can import sklearn later on.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
```

Get the Data

```
Loan_data= pd.read_csv('loan_data.csv')
```

Check out the info(), head(), and describe() methods on loans.

```
Loan_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   credit.policy      9578 non-null   int64
 1   purpose            9578 non-null   object
 2   int.rate           9578 non-null   float64
 3   installment        9578 non-null   float64
 4   log.annual.inc     9578 non-null   float64
 5   dti                9578 non-null   float64
 6   fico               9578 non-null   int64
 7   days.with.cr.line  9578 non-null   float64
 8   revol.bal          9578 non-null   int64
 9   revol.util         9578 non-null   float64
 10  inq.last.6mths     9578 non-null   int64
 11  delinq.2yrs        9578 non-null   int64
 12  pub.rec            9578 non-null   int64
 13  not.fully.paid     9578 non-null   int64
dtypes: float64(6), int64(7), object(1)
memory usage: 1.0+ MB
```

```
Loan_data.head()
```

	credit.policy	purpose	int.rate	installment	log.annual.inc	d
0	1	debt_consolidation	0.1189	829.10	11.350407	19.
1	1	credit_card	0.1071	228.22	11.082143	14.
2	1	debt_consolidation	0.1357	366.86	10.373491	11.
3	1	debt_consolidation	0.1008	162.34	11.350407	8.
4	1	credit_card	0.1426	102.92	11.299732	14.

```
Loan_data.describe()
```

	credit.policy	int.rate	installment	log.annual.inc	dti	
count	9578.000000	9578.000000	9578.000000	9578.000000	9578.000000	9
mean	0.804970	0.122640	319.089413	10.932117	12.606679	
std	0.396245	0.026847	207.071301	0.614813	6.883970	
min	0.000000	0.060000	15.670000	7.547502	0.000000	
25%	1.000000	0.103900	163.770000	10.558414	7.212500	
50%	1.000000	0.122100	268.950000	10.928884	12.665000	
75%	1.000000	0.140700	432.762500	11.291293	17.950000	
max	1.000000	0.216400	940.140000	14.528354	29.960000	

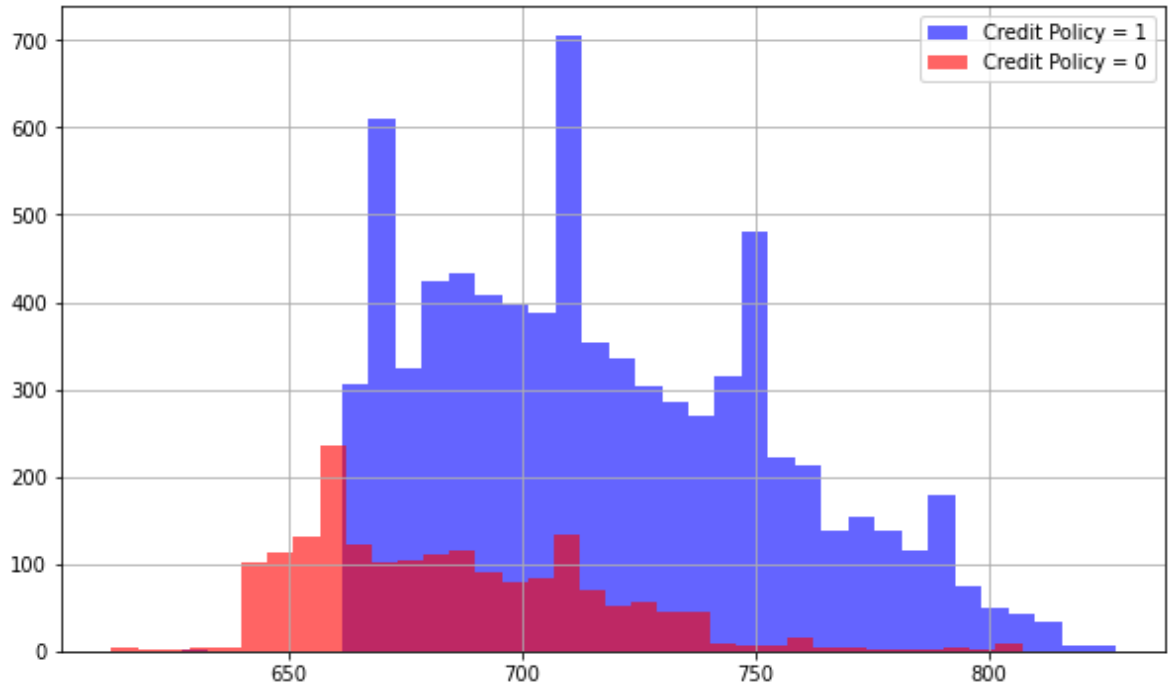
Exploratory Data Analysis

I used seaborn and pandas built-in plotting capabilities.

An histogram of two FICO distributions on top of each other, one for each credit.policy outcome.

```
plt.figure(figsize=(10,6))
Loan_data[Loan_data['credit.policy']==1]['fico'].hist(bins=35, color='blue',
                                                      label='Credit Policy
                                                      alpha=0.6)
Loan_data[Loan_data['credit.policy']==0]['fico'].hist(bins=35, color='red',
                                                      label='Credit Policy
                                                      alpha=0.6)
plt.legend()
```

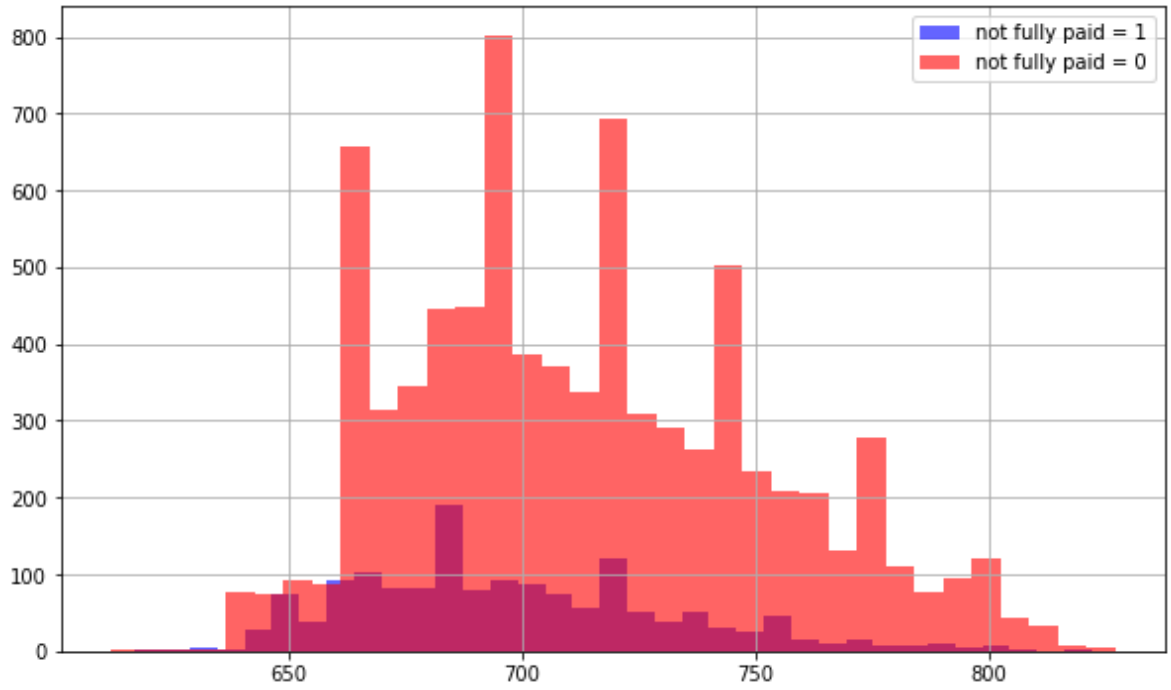
```
<matplotlib.legend.Legend at 0x7f180a570e10>
```



Histogram for not.fully.paid column.

```
plt.figure(figsize=(10,6))
Loan_data[Loan_data['not.fully.paid']==1]['fico'].hist(bins=35, color='blue'
                                                       label='not fully pai
                                                       alpha=0.6)
Loan_data[Loan_data['not.fully.paid']==0]['fico'].hist(bins=35, color='red',
                                                       label='not fully pai
                                                       alpha=0.6)
plt.legend()
```

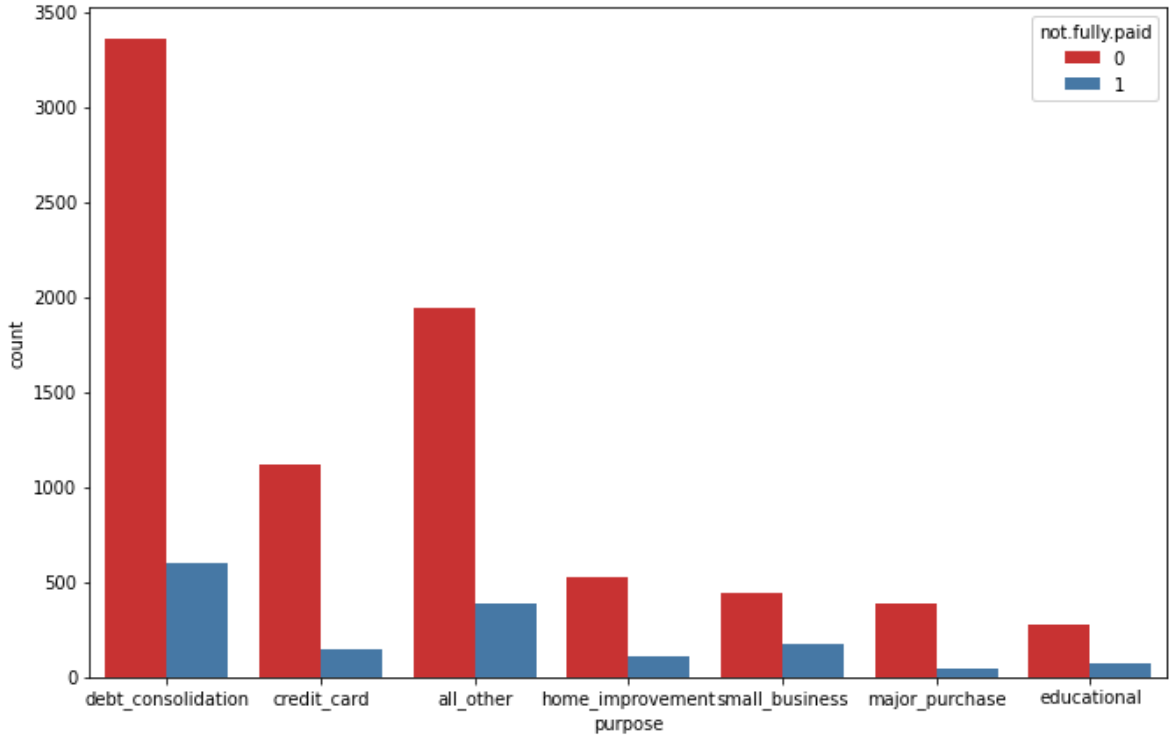
```
<matplotlib.legend.Legend at 0x7f180a99da90>
```



Countplot using seaborn showing the counts of loans by purpose, with the color hue defined by not.fully.paid. From here we can see that debt consolidation seem to be the major reason for wanting a loan, followed by credit card and other reasons. Other reasons are much lower on the list and this is pretty much obvious from the display. Also the ratio of fully paid and not fully paid looks pretty similar in all the ranges

```
plt.figure(figsize=(11,7))
sns.countplot(x='purpose', hue='not.fully.paid', data=Loan_data, palette='Se
```

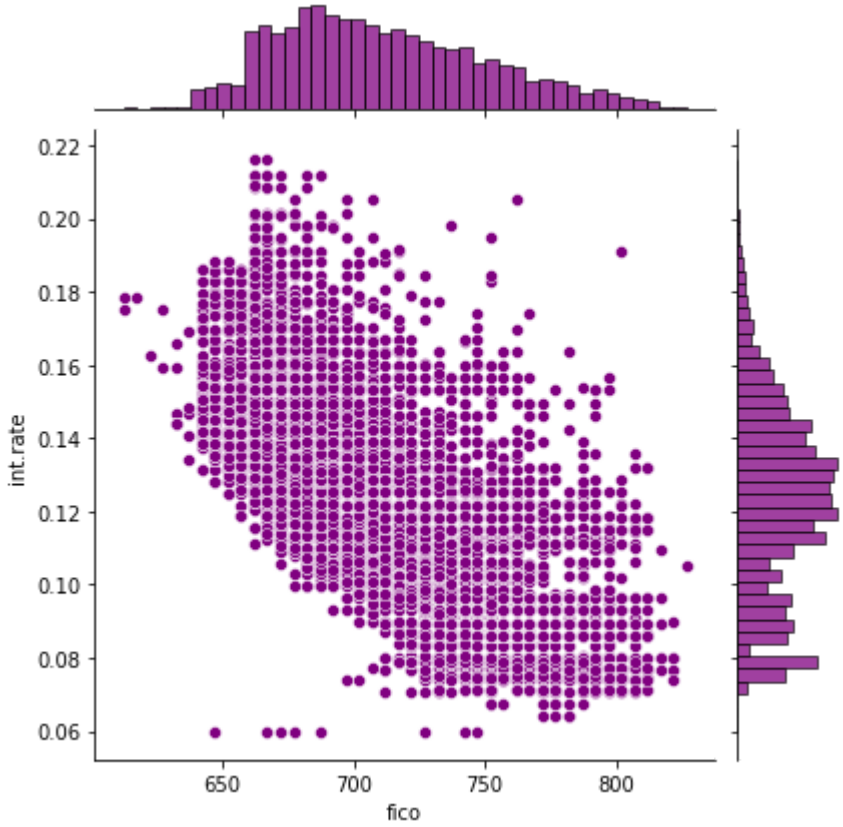
```
<matplotlib.axes._subplots.AxesSubplot at 0x7f180a9fac10>
```



The trend between FICO score and interest rate. As the Fico score increases the interest rate decreases and vice versa, this puts the urgency to quickly pay off the loan.

```
sns.jointplot(x='fico', y='int.rate', data=Loan_data, color = 'purple')
```

```
<seaborn.axisgrid.JointGrid at 0x7f18093490d0>
```

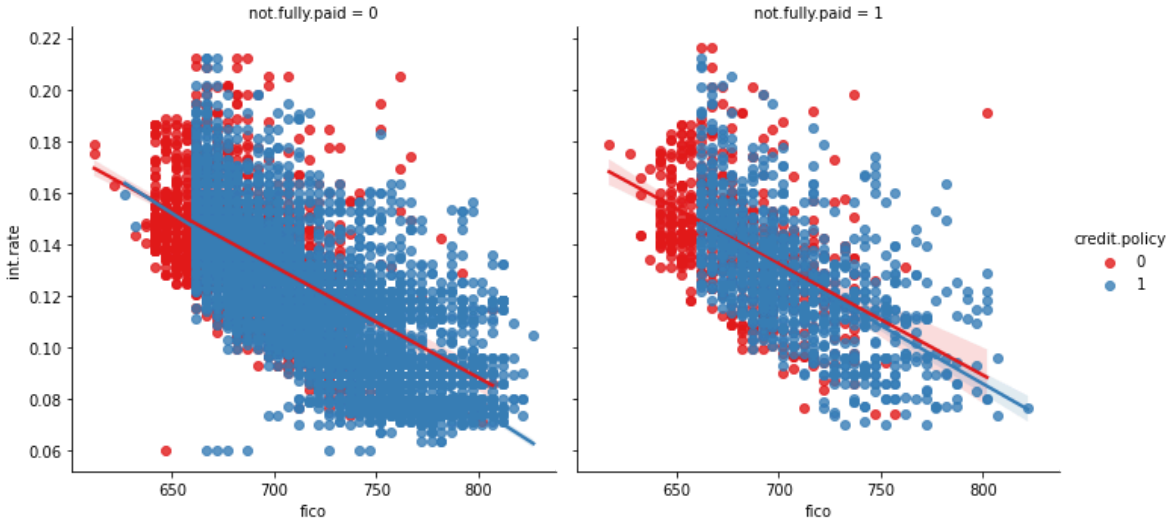


lmplots to see if the trend differed between not.fully.paid and credit.policy.

The behaviour was relatively the same, whether or not it was fully paid off or not, and whether or not they were denied the credit policy

```
plt.figure(figsize=(11,7))
sns.lmplot(y='int.rate', x= 'fico' , data = Loan_data, hue= 'credit.policy',
           col='not.fully.paid',palette='Set1')
```

```
<seaborn.axisgrid.FacetGrid at 0x7f1804f90710>
<Figure size 792x504 with 0 Axes>
```



Setting up the Data

Data for our Random Forest Classification Model!

```
Loan_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   credit.policy      9578 non-null   int64
 1   purpose            9578 non-null   object
 2   int.rate           9578 non-null   float64
 3   installment        9578 non-null   float64
 4   log.annual.inc     9578 non-null   float64
 5   dti                9578 non-null   float64
 6   fico               9578 non-null   int64
 7   days.with.cr.line  9578 non-null   float64
 8   revol.bal          9578 non-null   int64
 9   revol.util         9578 non-null   float64
 10  inq.last.6mths     9578 non-null   int64
 11  delinq.2yrs        9578 non-null   int64
 12  pub.rec            9578 non-null   int64
 13  not.fully.paid     9578 non-null   int64
dtypes: float64(6), int64(7), object(1)
memory usage: 1.0+ MB
```

Categorical Features

Notice that the **purpose** column as categorical Data types

That means I need to transform them using dummy variables so sklearn will be able to understand them. I will do this using 'pd.get_dummies'.

The way of dealing with these columns can be expanded to multiple categorical features if necessary.

** I created a list of 1 element containing the string 'purpose' and called this list cat_feats.**

```
Cat_feats = ['purpose']
```

Now use pd.get_dummies(loans,columns=cat_feats,drop_first=True) to create a fixed larger dataframe that has new feature columns with dummy variables. Set this dataframe as Final_data.

```
Final_data=pd.get_dummies(Loan_data, columns=Cat_feats, drop_first=True)
```

```
Final_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 19 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   credit.policy              9578 non-null   int64
 1   int.rate                   9578 non-null   float64
 2   installment                9578 non-null   float64
 3   log.annual.inc             9578 non-null   float64
 4   dti                        9578 non-null   float64
 5   fico                       9578 non-null   int64
 6   days.with.cr.line          9578 non-null   float64
 7   revol.bal                  9578 non-null   int64
 8   revol.util                 9578 non-null   float64
 9   inq.last.6mths             9578 non-null   int64
 10  delinq.2yrs                9578 non-null   int64
 11  pub.rec                    9578 non-null   int64
 12  not.fully.paid             9578 non-null   int64
 13  purpose_credit_card        9578 non-null   uint8
 14  purpose_debt_consolidation 9578 non-null   uint8
 15  purpose_educational        9578 non-null   uint8
 16  purpose_home_improvement   9578 non-null   uint8
 17  purpose_major_purchase     9578 non-null   uint8
 18  purpose_small_business     9578 non-null   uint8
dtypes: float64(6), int64(7), uint8(6)
memory usage: 1.0 MB
```

Train Test Split

Now its time to split the data into a training set and a testing set!

** Use sklearn to split the data into a training set and a testing set.**

```
from sklearn.model_selection import train_test_split
```

```
X=Final_data.drop('not.fully.paid', axis=1)
y = Final_data['not.fully.paid']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran
```

Training a Decision Tree Model

Import DecisionTreeClassifier

```
from sklearn.tree import DecisionTreeClassifier
```

** I created an instance of DecisionTreeClassifier(), Dtree and fit it to the training data.**

```
Dtree= DecisionTreeClassifier()
```

```
Dtree.fit(X_train,y_train)
```

```
DecisionTreeClassifier()
```

Predictions and Evaluation of Decision Tree

Create predictions from the test set and create a classification report and a confusion matrix.

```
Predictions =Dtree.predict(X_test)
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
print(classification_report(y_test, Predictions))
```

```
              precision    recall  f1-score   support

           0       0.85      0.82      0.84      2431
           1       0.19      0.22      0.20       443

    accuracy                           0.73      2874
   macro avg       0.52      0.52      0.52      2874
weighted avg       0.75      0.73      0.74      2874
```

```
print(confusion_matrix(y_test,Predictions))
```

```
[[2003  428]
 [ 344   99]]
```

Training the Random Forest model

Now its time to train our model!

Create an instance of the RandomForestClassifier class and fit it to our training data from the previous step.

```
from sklearn.ensemble import RandomForestClassifier
```

```
Rfc = RandomForestClassifier(n_estimators=300)
```

```
Rfc.fit(X_train,y_train)
```

```
RandomForestClassifier(n_estimators=300)
```

Predictions and Evaluation

Let's predict off the y_test values and evaluate our model.

** Predict the class of not.fully.paid for the X_test data.**

```
RPredictions = Rfc.predict(X_test)
```

Now create a classification report from the results. Do you get anything strange or some sort of warning?

```
print(classification_report(y_test, RPredictions))
```

Show hidden output